Automata-Based Smart Metro Ticketing and Journey Validation System

1. What is this project?

This project is a small simulation of a metro ticketing system. A passenger picks a starting station and a destination, pays the fare, and gets a digital ticket (a barcode/QR code) on their phone. The ticket is scanned at the entry gate and again at the exit gate. The system decides whether the journey is valid.

The special part is that the whole system is built using **Automata Theory**. Every step of a journey (booking, entering, exiting, paying extra) is treated as a **state**, and every event (pay, scan, exit) is treated as an **input symbol**. The system moves from one state to another based on the input, and at the end we can say whether the sequence of events was **accepted** (valid) or **rejected** (invalid).

The project shows that theoretical ideas such as DFA, NFA, regular expressions, minimization, Moore and Mealy machines can be used to model a real-life system.

---

2. What the system can do

1. Let a passenger choose a source and a destination station on a screen.
2. Calculate and show the fare.
3. Accept cash payment: the passenger inserts an amount, the system checks it against the fare, asks for more if it is too little, and returns any change.
4. Generate a signed digital ticket (QR/barcode).
5. Validate the ticket at the entry gate.
6. Validate the ticket at the exit gate.
7. Detect and handle **over-travelling** (going beyond the paid destination).
8. Detect and handle **over-time** (staying inside longer than any genuine journey needs, for example riding a full extra loop).
9. Allow **early exit** (leaving before the paid destination).
10. Reject forged, edited, reused, or wrongly used tickets.
11. Let the user type any sequence of events and see whether the automaton accepts it, with every state change shown.
12. Show the automata theory side: transition tables, regular expressions, NFA to DFA conversion, DFA minimization, and Moore/Mealy outputs.

---

3. The metro network

To keep the project simple and finishable, we use a **circular (round-trip) line**:

- **One loop with 8 stations**, numbered 0 to 7. The route is 0 → 1 → 2 → … → 7 → 0 → 1 → …
- Trains run in **one direction only** and never reverse. There is no terminus (no last station where the train turns back), so there is no forced exit anywhere. A passenger can get off at any station.
- Every station has **one global ID** (for example 000 to 007).
- **Distance is forward distance.** The distance from station A to station X is the number of stops going forward from A, wrapping around from 7 back to 0:

```
distance(src, x) = (x - src) mod 8
```

Examples: distance(2, 5) = 3, and distance(6, 1) = 3 (the route wraps around 7 → 0 → 1).

- **Fare depends on forward distance**, using simple slabs:

Stations travelled (forward)	Fare
1 to 2	₹10
3 to 5	₹20
6 to 7	₹30

The longest genuine journey is 7 stops. A journey of 8 stops would be a full loop back to your own source station, which is never a ticket you can buy.

- **Time:** we assume about **2.5 minutes between two stations** (including the stop). A full loop therefore takes **20 minutes**. These numbers are settings that can be changed.

Multiple lines and interchanges are left as future work.

---

4. The digital ticket (barcode / QR token)

Our first idea

Give every source-destination pair its own unique code.

Problem with that idea

Everyone travelling from A to D would get the **same** code. It could be copied, shared, or guessed, and it could not be marked as "used" without blocking the next genuine passenger.

Our final design

The ticket is a **signed journey token** made of four parts:

Part	Purpose
Pair code	Source station ID followed by destination ID. Tells the gate the route and the fare.
Transaction ID	A unique number for this one ticket, so it can be marked used or closed.
Timestamp	When the ticket was bought.
Signature	A short code made from the data using a secret key (HMAC). If anyone edits the ticket, the signature no longer matches and the ticket is rejected.

Example:

```
M1|T48213|03|07|1730500000.9fa3c21b
```

So our original "unique code for each pair" idea is still there, as the first part of the token. The other parts fix its weaknesses.

Two different checks at a gate

- **Is the ticket genuine?** The signature answers this by itself.

- **Has it already been used, and when did the passenger enter?** This needs a shared record. In this project, a simple dictionary or SQLite table plays the role of the central server. For each ticket it stores the current state, the source, the destination, the paid fare, and the **entry time** (recorded when the entry gate accepts the scan). The token's own timestamp only says when the ticket was bought, so it cannot be used for the time limit.

---

5. How a journey works

Scanning rules

- A passenger scans **only at the entry gate and the exit gate**. There is no scanning at stations in between, just like a real metro.
- Each platform is a gated area. Trains only move forward, and a passenger cannot cross to another platform without leaving the paid area. Leaving means an exit scan, and the ticket closes. To travel again, they buy a new ticket.
- Entering again with a ticket that is already **inside** is rejected as a double entry.
- When the entry gate accepts a scan, the system records the **entry time**.

At the exit gate: three comparisons

When a passenger taps out, the gate knows:

- the source, destination and paid fare (from the token),
- the actual exit station (the gate's own location),
- whether the ticket is currently "inside", and when it entered (from the shared record).

It then checks, in this order:

1. **Time:** how long has the passenger been inside? If it is more than the time limit, the result is **over-time**.
2. **Destination:** is the exit station the destination? If yes, **normal exit**.
3. **Fare:** compare the forward distance (and fare) from the source to the exit station with what was paid. Less means **early exit**, more means **over-travel**.

Example: ticket from station 0 to station 3 (3 stops, ₹20 paid).

Passenger exits at	Forward distance from 0	Result
2	2 (₹10, less than ₹20)	Early exit
3	3 (the destination)	Normal exit
6	6 (₹30, more than ₹20)	Over-travel, excess ₹10
0, after a full loop	time inside is about 20 minutes	Over-time

Example with wrap-around: ticket from station 6 to station 1 is 3 stops (₹20). Exiting at station 7 is 1 stop, which is an early exit. Exiting at station 1 is the destination.

---

6. Early exit

If the passenger leaves before the destination, the gate opens and the ticket is closed.

- **Policy used:** no refund. The passenger chose to buy a longer ticket, so nothing is owed, and the metro has lost nothing.
- Leaving at the source station almost immediately after entering (distance 0) is treated the same way.
- A partial refund is a possible future extension.

---

7. Over-travelling and over-time

Both end the same way: the gate locks and the passenger pays before leaving.

Over-travelling (went further than paid)

1. The gate **stays locked**.
2. The screen shows "Pay excess ₹__", where excess = fare for the forward distance to the exit station minus the fare paid.
3. The ticket moves to an **Excess Pending** state. It is not destroyed, and the passenger is still counted as inside.
4. The passenger pays the excess.
5. The ticket moves to **Exited**, the gate opens and the ticket closes.

If the passenger taps again without paying, nothing changes and the gate stays locked.

Over-time (stayed inside too long)

Handled the same way, using the same Excess Pending state, with a fixed **over-time charge** (₹20 by default, a setting) instead of a distance-based excess.

Over-travelling, over-time and early exit are not special extras. They are the normal ways the automaton can leave the "Inside" state, which is why the automata model fits so well.

---

8. The time limit (why it exists and how it is set)

The problem it solves

On a one-direction loop, going further than the paid stop is already caught by the fare comparison. But the loop **wraps around**, so one problem remains: exiting at the source station could mean "entered and left immediately" (0 stops) or "rode the entire loop" (8 stops). Both end at the same station. Exiting at any other station has the same ambiguity (for example, station 3 after 3 stops looks like station 3 after 11 stops). Only **time** can tell these apart, because a full extra loop takes much longer than any genuine journey.

The rule

We use one **flat time limit** for all tickets:

```
limit = time for the longest genuine journey (7 stops) + a small buffer
      = 7 x 2.5 min + 1.5 min
      = 19 minutes
```

- The **longest genuine journey** takes 17.5 minutes, so nobody travelling honestly is penalised.
- The **buffer** covers real delays such as gate queues or a slightly late train. It must be **shorter than the time between two stations** (2.5 minutes). If it were longer, a passenger could squeeze in one extra stop.
- A full loop takes **20 minutes**, which is more than the 19-minute limit. So any journey that includes a full extra loop is always caught.
- Because the limit is below a full loop, the exit station can only mean one thing when the time check passes. This removes the wrap-around ambiguity.

Why a flat limit is enough here

Because trains only go forward and a passenger cannot turn around, the fare comparison already handles everything except the full loop. The time limit only has to catch the full loop, so one flat number does the job. A limit that changes with each ticket is not needed.

---

9. Known limitations (stated honestly)

- The system cannot see what happens between the entry and exit scans. This is handled by the design rules: forward-only trains, gated platforms, the fare comparison, and the time limit.
- The time limit uses time as a stand-in for distance. A passenger who genuinely waits a long time on a platform can be flagged as over-time. The buffer covers small delays only.
- The time values (2.5 minutes between stations, 19 minute limit) are assumptions for the simulation.
- No real camera scanning. A "scanner box" where the token is pasted is used for the demo, plus a simulated clock for time inside.
- Only one circular line, no interchanges, and trains in one direction only.
- No refunds. The over-time charge is a fixed amount.

---

10. The automata used in the project

10.1 Why a classifier is needed

A DFA works with a fixed set of symbols and cannot compare numbers such as station IDs, fares or minutes. So we split the job:

1. A **classifier** (normal code) reads the token and the scan details, checks the signature, compares distances and time, and outputs **one symbol**.
2. The **DFA** takes that symbol and changes state.

The DFA models the **control flow** of the ticket. The classifier and fare table are a supporting data layer. This keeps the DFA small and the same whether the metro has 8 stations or 500.

(We also considered building a separate DFA for every ticket. We rejected it because the alphabet and the number of automata grow with the network, so it does not scale.)

10.2 Booking DFA (the kiosk)

Models the kiosk steps from idle to ticket generation.

States

State	Meaning
q0	Idle
q1	Source selected
q2	Destination selected
q3	Fare displayed
q4	Payment pending
q5	Paid
q6	Ticket generated (accepting)
qd	Dead state (invalid action)

Symbols: `src, dst, confirm, pay_ok, pay_fail, gen, cancel`

How payment produces a symbol: the user does not choose the payment result. The kiosk takes an inserted cash amount and the system decides:

- Amount inserted is **at least the fare** → `pay_ok`, and any change is returned.
- Amount inserted is **less than the fare** → `pay_fail`. The state stays at q4, the user inserts more cash, and the check is repeated.

Example: a ticket from 0 to 3 costs ₹20. Inserting ₹10 gives `pay_fail`, and inserting another ₹10 brings the total to ₹20 and gives `pay_ok`. This is why the regex contains `pay_fail*`.

Transition table

State	src	dst	confirm	pay_ok	pay_fail	gen	cancel
q0	q1	qd	qd	qd	qd	qd	q0
q1	q1	q2	qd	qd	qd	qd	q0
q2	q1	q2	q3	qd	qd	qd	q0
q3	qd	qd	q4	qd	qd	qd	q0
q4	qd	qd	qd	q5	q4	qd	q0
q5	qd	qd	qd	qd	qd	q6	qd
q6	qd	qd	qd	qd	qd	qd	qd

Regular expression of accepted sequences

```
src (src | dst)* dst confirm pay_fail* pay_ok gen
```

10.3 Journey DFA (the core of the project)

Models the life of one ticket from issue to exit.

States

State	Meaning
J0	Ticket issued, not used yet
J1	Inside the system
JE	Excess charge pending (gate locked)
J2	Exited normally (accepting)
J3	Exited early (accepting)
JT	Rejected / trap state

Symbols (produced by the classifier)

Symbol	Meaning
E	Valid entry scan at the source station
D	Exit at the destination
X	Early exit (distance to exit station is less than the distance paid for)
O	Over-travel (distance to exit station is more than the distance paid for)
T	Over-time (time inside is more than the time limit)
P	Excess or over-time charge paid
W	Wrong use: forged token, wrong entry station, double entry, or exit without entry
R	Reuse of a ticket that is already closed

Transition table

State	E	D	X	O	T	P	W	R
J0	J1	JT	JT	JT	JT	JT	JT	JT
J1	JT	J2	J3	JE	JE	JT	JT	JT
JE	JT	JT	JT	JE	JE	J2	JT	JT
J2	JT	JT	JT	JT	JT	JT	JT	JT
J3	JT	JT	JT	JT	JT	JT	JT	JT
JT	JT	JT	JT	JT	JT	JT	JT	JT

Regular expression of accepted journeys

```
E (D | X | (O | T)+ P)
```

Example sequences

Sequence	Result	Meaning
ED	Accept	Normal journey
EX	Accept	Early exit
EOP	Accept	Over-travel, then paid the excess
ETP	Accept	Over-time, then paid the charge
EOOP	Accept	Tried to exit twice, then paid
EO	Reject	Stuck at the locked gate
ET	Reject	Stuck at the locked gate
D	Reject	Exit without entry
EE	Reject	Double entry
EDR	Reject	Reuse after exit

10.4 NFA and conversion to DFA

An NFA is used where a choice exists. The example in this project is the **payment method**: after the passenger confirms the fare, the machine can branch to UPI, card or cash. All the branches merge into the **Paid** state. (This NFA is a theory demonstration of choice and merging. The kiosk itself in this project accepts cash only.)

The project shows:

1. The NFA diagram and its transition table.
2. **Subset construction**, showing which set of NFA states becomes which DFA state.
3. The resulting equivalent DFA.

This proves that an NFA and a DFA can describe exactly the same behaviour.

10.5 DFA minimization

We build a deliberately slightly redundant version of the Booking DFA, for example with separate states `PaidCash` and `PaidUPI` that behave identically. We then apply the **table-filling (Myhill–Nerode) method**:

1. Mark every pair of one accepting and one non-accepting state as distinguishable.
2. Repeatedly mark pairs whose transitions lead to an already marked pair.
3. Pairs that stay unmarked are equivalent and are merged.

The app shows the before and after automata and the marking table, for example "8 states reduced to 7 states".

10.6 Moore machine (output depends on the state)

State	Output
J0	Ticket ready
J1	Entry gate open
JE	Gate LOCKED, show "Pay excess ₹" **or "Pay over-time charge ₹"**
J2	Exit gate open, "Thank you"
J3	Exit gate open, "Early exit recorded"
JT	Gate LOCKED, alarm

10.7 Mealy machine (output depends on the state and the input)

Written as `state --input/output--> next state`:

- `J0 --E/Open entry gate, record entry time--> J1`
- `J1 --D/Open exit gate--> J2`
- `J1 --X/Open exit gate, record early exit--> J3`
- `J1 --O/Lock gate, show excess fare--> JE`

- `J1 --T/Lock gate, show over-time charge--> JE`
- `JE --P/Open exit gate--> J2`

Both machines describe the same gate behaviour in two different styles.

---

11. The classifier rules

For every scan, the classifier produces one symbol. `dist(a, b)` means `(b - a) mod 8`. `time_inside` is the exit time minus the recorded entry time.

Entry gate

- Signature invalid → `W`
- Ticket already closed → `R`
- Ticket unused and scan station is the source → `E` (and the entry time is recorded)
- Otherwise (wrong station, or already inside) → `W`

Exit gate

- Signature invalid → `W`
- Ticket already closed → `R`
- Ticket not marked inside → `W`
- `time_inside` greater than the limit (19 minutes) → `T`
- Exit station is the destination → `D`
- `dist(source, exit station)` less than `dist(source, destination)` → `X`
- Otherwise → `O`

When the passenger pays the excess or over-time charge, the system feeds `P` into the Journey DFA.

---

12. The screens of the application

The application has four screens. The first two show the real system (booking and gates), and the other two show the automata theory behind it.

12.1 Kiosk screen (booking a ticket)

What the user sees

- Two dropdowns for the source and destination station.
- A **Show Fare** button, and a fare box that fills in once both stations are chosen.
- A **Proceed to Pay** button.
- A cash insertion area: an amount box, or note buttons for ₹10 / ₹20 / ₹50, with a running **"inserted so far"** total.
- A message that says either "₹__ more needed" or "Change to return: ₹__".
- A **Generate Ticket** button, and a **Cancel** button that is always available.
- After generation, the ticket appears as a QR image or as the text token.
- A side panel with the Booking DFA drawn as a diagram, where the **current state is highlighted**, and a small list showing the symbols fed in so far.

What happens behind the scenes Every click is one input symbol for the Booking DFA. The user never chooses the payment result. Each time cash is inserted, the system compares the total inserted with the fare and produces `pay_ok` or `pay_fail` itself.

User action	Symbol	State change
Pick a source	`src`	q0 → q1
Pick a destination	`dst`	q1 → q2
Press Show Fare	`confirm`	q2 → q3
Press Proceed to Pay	`confirm`	q3 → q4
Insert cash, total less than the fare	`pay_fail`	q4 → q4 (stays, insert more)
Insert cash, total at least the fare	`pay_ok`	q4 → q5 (change is returned)
Press Generate Ticket	`gen`	q5 → q6 (accepting)
Press Cancel	`cancel`	back to q0

If the user changes the source or destination after choosing, that simply feeds `src` or `dst` again, which is why the table allows it. Source and destination cannot be the same station, since a ticket needs at least one stop.

Example: a ticket from 0 to 3 costs ₹20. The user inserts ₹10, the screen says "₹10 more needed" and the state stays at q4. The user inserts another ₹10, the total reaches ₹20, the state moves to q5, and Generate Ticket becomes available.

Why it matters The highlighted state makes the DFA visible. A viewer can watch "not enough cash, still at q4" and "paid, now at q5" as they happen, and the accepted sequence matches the regex `src (src | dst)* dst confirm pay_fail* pay_ok gen`.

12.2 Gate simulator (entry and exit scanning)

What the user sees

Inputs:

- **Station:** which station this gate is at.
- **Gate type:** entry or exit.
- **Token box:** where the ticket token is pasted (a stand-in for a camera).
- **Minutes since entry:** the simulated clock. It matters at exit gates. Real waiting is replaced by typing a number, so a 20-minute loop can be shown in two seconds.
- A **Scan** button.

Outputs after pressing Scan:

- The **symbol** the classifier produced (`E`, `D`, `X`, `O`, `T`, `W` or `R`).
- The **state change**, for example `J1 → J2`.
- The **Moore output** (what the new state says) and the **Mealy output** (what the transition says).
- A **gate indicator**: green OPEN or red LOCKED.
- A message such as "Pay excess ₹10".
- A **Pay** button, which appears only when the gate is locked for an excess or over-time charge.
- A small **ticket status box** showing the current state of this ticket.
- An optional **history line** listing the symbols for the current ticket so far (for example `E O`), so it can be copied straight into the string tester.

What happens behind the scenes Scan runs `process_scan`. The classifier checks the signature, looks up the ticket in the shared record, compares distances and time, and produces one symbol. That symbol goes into the Journey DFA, the new state is saved, and the outputs are shown. Pressing Pay feeds `P` into the DFA.

A demo walk-through (ticket from station 0 to station 3, ₹20 paid)

Step	Input	Symbol	Result
1	Entry gate at station 0	`E`	J0 → J1, gate opens, entry time saved
2a	Exit at station 3, 8 min	`D`	J1 → J2, gate opens
2b	Exit at station 2, 5 min	`X`	J1 → J3, gate opens, early exit
2c	Exit at station 6, 15 min	`O`	J1 → JE, gate locked, "Pay excess ₹10"
2d	Exit at station 0, 20 min	`T`	J1 → JE, gate locked, "Pay over-time ₹20"
3	Press Pay (after 2c or 2d)	`P`	JE → J2, gate opens

Other things worth showing: pasting an edited token gives `W` (rejected), scanning again after a normal exit gives `R` (reuse), and trying the entry gate at the wrong station gives `W`.

Why it matters This is the heart of the project. It proves that the over-travel, early exit and time limit rules actually work on real tickets, and it shows the classifier and the DFA cooperating.

12.3 String tester (pure automata mode)

What the user sees

- A text box to type a sequence of symbols, such as `EOP`.
- A selector for which automaton to test: Journey DFA or Booking DFA.
- A **Run** button.
- A **trace table** listing every step, and a big **ACCEPTED** or **REJECTED** verdict.

Example: typing `EOP` on the Journey DFA

Step	Symbol read	From	To
1	E	J0	J1
2	O	J1	JE
3	P	JE	J2

The final state is J2, which is accepting, so the verdict is ACCEPTED.

Example: typing `EO` The trace ends in JE, which is not accepting, so the verdict is REJECTED. The screen can add a note such as "ended in a non-accepting state". For something like `EE`, the trace falls into the trap state JT, and the screen can say the string was rejected because the DFA entered the dead state.

Why it matters It lets the examiner type any sequence they like and see the DFA decide, with no stations, tokens or time involved. It is the clearest way to show the language the DFA accepts. The screen can also show whether the string matches the regular expression, so the regex and the DFA are seen to agree.

12.4 Automata viewer (the theory tab)

This screen collects all the theory in one place, as a set of tabs or sections.

Tables and regexes

- The transition tables for the Booking DFA and the Journey DFA.
- The regular expression for each, next to the diagram.
- The Moore and Mealy tables for the gate.

NFA to DFA

- The payment-method NFA, as a diagram and a table.
- The subset-construction steps, shown as a table of "which set of NFA states becomes which DFA state".
- The resulting DFA, with a note that both accept the same strings.

Minimize button Pressing it takes the deliberately redundant Booking DFA (for example with `PaidCash` and `PaidUPI`) and shows:

1. The **before** automaton and its state count (for example 8 states).
2. The **marking table** built round by round, with the pairs of distinguishable states marked.
3. The pairs left unmarked, which are the equivalent states.
4. The **after** automaton with those states merged (for example 7 states).

Why it matters This is what answers the topics promised in the abstract: NFA, subset construction, minimization, Moore and Mealy. Without this screen, those parts exist only in the report. With it, they are shown live.

12.5 How the screens connect

The kiosk **creates** a ticket, and the gate simulator **uses** that ticket, so the demo naturally flows from one to the other by copying the token. Every symbol produced at a gate is also a valid input for the string tester, so you can show "these scans produced `E` then `O` then `P`, and here is the same sequence run in the tester". The automata viewer then shows the theory those two screens are built on.

---

13. Test cases

Sequence / situation	Expected result
E D	Accept
E X	Accept
E O P	Accept
E T P	Accept
E O O P	Accept
E O	Reject
E T	Reject
D (no entry)	Reject
E E	Reject
E D R	Reject
Edited token	Reject (W)
Wrong entry station	Reject (W)

Situation tests for the classifier (ticket from 0 to 3, ₹20 paid):

Exit station	Minutes inside	Expected symbol
3	8	D
2	5	X
6	15	O
0	20	T (full loop)
3	25	T (too long, even at the destination)
0	1	X (entered and left at once)

Ticket from 6 to 1 (wrap-around): exit at 1 after 8 minutes gives `D`, exit at 7 after 3 minutes gives `X`.

---

14. Technology

- **Language:** Python
- **Interface:** Streamlit (quick web interface) or Tkinter
- **Token security:** `hmac` and `hashlib` from the Python standard library
- **QR code:** `qrcode` library (optional)
- **Ticket records:** a Python dictionary or SQLite
- **Diagrams:** draw.io or graphviz

Settings (kept in one place so they are easy to change)

```
NUM_STATIONS = 8
MINUTES_PER_STATION = 2.5
BUFFER_MINUTES = 1.5        # must stay below MINUTES_PER_STATION
TIME_LIMIT = (NUM_STATIONS - 1) * MINUTES_PER_STATION + BUFFER_MINUTES   # 19 minutes
OVERTIME_CHARGE = 20
```

Suggested file structure

```
metro_project/
├── config.py          # the settings above
├── stations.py        # station list, distance and fare functions
├── token_utils.py     # create_token(), verify_token(), QR generation
├── automata.py        # DFA, NFA, Moore, Mealy, subset construction, minimization
├── booking.py         # Booking DFA definition
├── journey.py         # classifier and Journey DFA
├── app.py             # user interface
└── tests.py           # valid and invalid sequences
```

---

15. Future work

- Multiple lines with interchanges (graph and shortest path for fare), and trains in both directions.
- A time limit that scales with the length of the ticket.
- Partial refunds for early exit and distance-based penalty fares.
- Real camera scanning of QR codes.
- Smart cards and monthly passes.

---

16. Keywords

Automata Theory, DFA, NFA, Regular Expressions, Subset Construction, DFA Minimization, Moore Machine, Mealy Machine, Digital Ticketing, QR/Barcode, Signed Token, Journey Validation, Circular Metro Line, Time Limit.